Sujet de TP 4 : Cloud versus Edge AI

L'**IA Cloud** désigne l'utilisation de serveurs distants haute performance pour stocker, traiter et analyser des données provenant d'objets connectés. Contrairement à l'IA embarquée, cette approche centralise l'intelligence sur des plateformes comme **ThingSpeak**, permettant d'exécuter des algorithmes complexes qui dépassent les capacités matérielles des appareils locaux.

Les avantages stratégiques de cette architecture incluent :

- **Puissance de calcul massive :** Capacité d'exécuter des modèles de Deep Learning gourmands en ressources et de traiter des pétaoctets de données.
- **Agrégation de données (Big Data) :** Centralisation des informations provenant de milliers de capteurs pour identifier des corrélations globales.
- **Stockage historique illimité :** Conservation de données sur le long terme pour l'entraînement et l'amélioration continue des modèles.
- **Mises à jour centralisées :** Possibilité de modifier un algorithme instantanément pour l'ensemble du parc d'appareils sans intervention physique.
- **Visualisation avancée :** Utilisation d'outils comme MATLAB au sein de ThingSpeak pour générer des tableaux de bord et des analyses statistiques complexes.

L'**IA embarquée** désigne l'intégration de capacités d'intelligence artificielle au sein d'appareils ou de systèmes, souvent appelés **applications de bord** (*edge AI*). Cela permet d'exécuter des tâches de machine learning localement, sans nécessiter de ressources de calcul externes telles que les services cloud.

Les principaux avantages de cette approche sont les suivants :

- **Latence réduite :** Grâce à la proximité de l'unité de traitement par rapport à la tâche.
- **Faible consommation de bande passante :** Besoins minimaux en transmission de données.
- **Faible consommation d'énergie :** Optimisée pour l'efficacité énergétique.
- **Indépendance vis-à-vis du réseau :** Fonctionne sans connectivité Internet constante.
- **Haute performance :** Capable de gérer efficacement des tâches exigeantes.

Pour maximiser l'efficacité d'un système intelligent, il ne s'agit pas de choisir entre l'IA embarquée et le Cloud, mais de les faire collaborer. Cette approche, souvent orchestrée via des outils comme **MATLAB** et **ThingSpeak**, repose sur une répartition stratégique de l'intelligence. Le véritable avantage de ce parallèle réside dans la boucle de rétroaction entre les deux systèmes :

- **L'IA Embarquée agit :** Le capteur surveille une machine en temps réel et prend des micro-décisions de sécurité.
- **Le Cloud apprend :** Les données significatives sont envoyées à **ThingSpeak**. Là, des algorithmes MATLAB puissants analysent les données de milliers de machines pour détecter des modèles d'usure invisibles à l'échelle locale.
- **L'IA s'améliore :** Une fois le nouveau modèle optimisé dans le Cloud, il est renvoyé vers l'appareil embarqué sous forme de mise à jour. L'appareil devient alors plus intelligent grâce à l'expérience partagée de toute la flotte.

1. Neural Network dans le Cloud MatWorks

L'objectif de cette étape est d'implémenter un réseau des neurones sur le Cloud de MatWorks. Pour cela, nous allons utiliser un modèle déjà entrainer avec Python/Tensorflow qui serait déployer sur le Cloud avec ThingSpeaks. En passant par Open Neural Network Exchange (ONNX), on utilise un format intermédiaire propre et standardisé : TensorFlow est d'abord converti en un graphe unifié, puis MATLAB lit ce graphe

simplifié. ONNX est indépendant des frameworks, centré sur l'inférence et basé sur des couches et opérations standardisées, ce qui le rend plus simple à interpréter. MATLAB manipule ainsi un réseau de neurones clair et structuré plutôt que de devoir gérer toute la complexité de l'écosystème TensorFlow.

ONNX est généralement plus fiable car il s'agit d'un format de modèle neutre et standardisé, spécialement conçu pour faciliter le transfert de modèles d'intelligence artificielle entre différents frameworks. Lors d'une importation directe d'un modèle TensorFlow vers MATLAB, le logiciel doit effectuer de nombreuses opérations complexes : analyser le graphe *SavedModel*, reconstruire les couches du réseau, traduire les opérations propres à TensorFlow en couches MATLAB et gérer les métadonnées spécifiques à Keras. De plus, les modèles TensorFlow contiennent souvent des éléments dépendants du framework, comme des opérations spécifiques, des couches personnalisées, des nœuds utiles uniquement pendant l'entraînement ou encore des formats de graphe variables selon les versions. Cette richesse fonctionnelle rend la conversion plus fragile et plus sujette aux erreurs. ONNX a été créé par Microsoft et Meta avec un objectif précis : assurer l'interopérabilité entre les outils d'intelligence artificielle. Il sert de langage commun permettant de transférer de manière fiable des modèles entraînés d'un environnement à un autre.

a. Exporter le modèle d'IA déjà entrainer en Python/Tensorflow. Par exemple, nous pouvons s'en servir des résultats de la question précèdent. N'oubliez pas de transférer le modèle TensorFlow ->ONNX->MatLab. L'exemple MNIST du TP1 serait implémenter de la façon suivante :

Python

```python
import tf2onnx
import tensorflow as tf

model = tf.keras.models.load_model("mnist_softmax_model")
spec = (tf.TensorSpec((None,784), tf.float32, name="input"),)
output_path = "mnist_softmax.onnx"

model_proto, _ = tf2onnx.convert.from_keras(model, input_signature=spec, output_path=output_path)
```

MATLAB

```MATLAB
model = importONNXNetwork("mnist_softmax.onnx", ...
    OutputLayerType="classification");
```

b. Implémenter un script MatLab qui tourne le réseau des neurones pour le problème MeteoStat en mode inférence. C'est-à-dire exécution d'une classification à partir du réseau déjà entrainer. Un exemple MNIST donnerait ceci

```matlab
% Load network
model = importONNXNetwork("mnist_softmax.onnx", OutputLayerType="classification");

% Read image
img = imread("digit.bmp");

if size(img,3) == 3
    img = rgb2gray(img);
end

img = im2single(img);
img = reshape(img, 1, []);

% Predict
scores = predict(model, img);

% Get digit
[prob, digit] = max(scores);
digit = digit - 1;

% Print
fprintf("Predicted digit: %d (confidence %.2f%%)\n", digit, prob*100);
```

c. Utiliser les données réelles obtenus sur la carte STM32 (voir TP2) et qui sont disponibles sur ThingSpeaks. Sauvegarder le résultat de la classification sur un canal spécifique.
d. Utiliser l'API Talkback pour faire afficher le résultat de la classification sur la carte STM32.

2. Neural Network dans la carte STM32N6

L'objectif de cette étape est d'implémenter un réseau des neurones sur la carte STM32N6. Un exemple d'implémentation d'un modèle simple est disponible sur le GitHub. Il sera nécessaire d'implémenter un code de réseau de neurones en mode inférence à partir des données acquises avec les capteurs de la carte fille.

a. Exporter le modèle d'IA déjà entrainer en Python/Tensorflow. Par exemple, nous pouvons s'en servir des résultats de la question précèdent. N'oubliez pas de transférer le modèle TensorFlow ->ONNX->C (via X Cube AI)
b. Implémenter un code C qui tourne le réseau des neurones pour le problème MeteoStat en mode inférence. C'est-à-dire exécution d'une classification à partir du réseau déjà entrainer.
c. Utiliser les données réelles acquises via les capteurs de la carte fille. Comparer ce résultat en termes de accuracy et temps d'exécutions avec la solution Cloud AI (ThingSpeaks).
d. Mesurer ou estimer la consommation de puissance de la carte lors d'une action de classification via le réseau implémenté. Pour cela, il faut désactiver les GPIOs et tout composant non essentiel. Préparer un programme témoins qui a toutes les étapes sauf l'exécution du modèle d'IA. Exécuter le programme témoin et le programme avec le réseau en mesurant le courant d'alimentation du jumper correspondant. (PS. Lisez bien la documentation de la carte avant cette procédure et venez discuter avec les charges de TP pour valider la manipulation).